

Relatório 11 – Lidando com Dados do Mundo Real

Edryck Freitas Nascimento

1. Introdução

O objetivo foi aprender sobre técnicas de *data mining* e *Machine Learning*, abordando desde a infraestrutura de *Data Warehousing* até algoritmos como k-NN, PCA e Q-Learning, métricas de avaliação, limpeza de dados e engenharia de recursos. A tarefa consistia em assistir às duas seções do curso *Machine Learning, Data Science and Generative AI with Python* e realizar um relatório sobre os conhecimentos adquiridos.

2. Desenvolvimento

2.1. Data Warehousing

Data Warehousing é uma estrutura centralizada de armazenamento de dados que irá conter diversas informações de diversas fontes, essa estrutura tem o desafio de coletar dados de diversas fontes, transformando-os em um esquema que permite consultar essas diferentes fontes de forma simultânea e conseguir *insights* por meio da análise de dados, utilizando essas fontes distintas.

Para fazer um *data warehousing* existem duas formas: ETL (*Extract, Transform, Load*) e ELT (*Extract, Load, Transform*).

2.1.1. ETL

É a maneira mais convencional de fazer *data warehousing*. Basicamente, é realizada a extração de dados brutos, em seguida, transforma todas essas informações em uma tabela de banco de dados estruturada preparando os dados antes do carregamento, por último, esses dados são carregados na *data warehousing*.

2.1.2. ELT

Ele permite aproveitar o poder dos bancos de dados distribuídos, construídos sobre Hadoop, como *Hive*, *Spark* ou *MapReduce*, e usá-los para realizar a transformação após o carregamento dos dados. A ideia aqui é extrair as informações, como era feito no ETL, depois carregá-las diretamente em um repositório de dados, para usar o poder do próprio repositório para realizar a transformação no local.

Ele acaba sendo mais vantajoso em situações em que está lidando com um enorme volume de dados.

2.1.3. ETL vs. ELT

A própria *data warehousing* é uma tabela real em um banco de dados, então à medida em que o volume de dados cresce, a etapa de transformação pode acabar se tornar um problema, a quantidade de processamento para realizar uma análise seria enorme, o que pode acabar se tornando um desafio de escalabilidade e estabilidade, quando utilizado o conceito de ETL. É nesse contexto que o conceito ELT se apresenta como uma alternativa escalável.

Enquanto no ETL a transformação acontece antes de chegar no *warehousing*, fora dele. No ELT, ela acontece dentro do *warehousing*, aproveitando o poder de processamento distribuído.

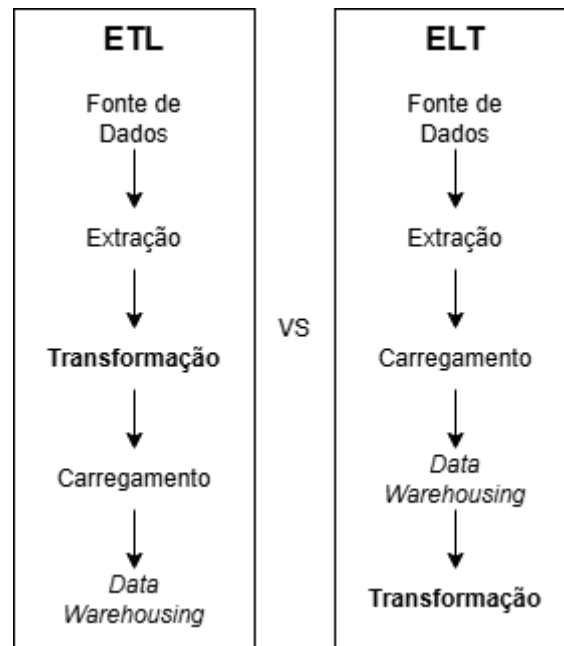


Figura 1. Comparação entre ETL e ELT. (autoria própria)

2.2. Algoritmos: k-NN, PCA e Q-Learning

Agora falando sobre algumas técnicas de aprendizado de máquina.

2.2.1. k-NN

k-vizinhos mais próximos (k-NN) é um dos mais simples para se começar, ele é uma das técnicas de aprendizado de máquina supervisionada simples, usada para a classificação e regressão.

Basicamente, a ideia de k-NN é classificar um novo ponto de dados através de uma análise dos vizinhos mais próximos deste novo ponto com base na métrica de distância. Apesar de ser fácil, o que está sendo feito é apenas pegar os vizinhos mais próximos em um gráfico de dispersão e depois deixar que eles “votem” em uma classificação para este novo ponto de dados.

2.2.2. PCA

Análise de Componentes Principais (PCA) se refere a uma técnica chamada decomposição de valores singulares (SVD). Muitos problemas podem ser considerados como tendo muitas dimensões diferentes, portanto, existem técnicas de redução de dimensionalidade para encontrar uma maneira de reduzir informações de alta dimensionalidade para informações de baixa dimensionalidade. Isso não só facilita a visualização e a classificação, como pode ser útil para coisas como compressão de dados.

A técnica PCA é um exemplo para redução de dimensionalidade, ela envolve matemática complexa (basicamente álgebra linear), ela pega um espaço de dados de alta dimensionalidade, encontra hiperplanos definidos por autovetores e projeta os dados nesses hiperplanos, que tornam novos eixos no espaço de dados de baixa dimensionalidade.

2.2.3. Q-Learning

É uma implementação de aprendizado por reforço onde a ideia é que você tem um agente que explora um espaço. Conforme explora esse espaço, ele aprende o valor de diferentes mudanças de estado em diferentes condições.

Enquanto o agente explora esse espaço, ele refina esses valores de recompensa para um determinado estado e pode usar esses valores de recompensa armazenados para escolher a melhor decisão a ser tomada, dada as condições atuais.

Começa com um conjunto de estados ambientais e os estados possíveis são as condições ao redor do agente, e tem um conjunto de ações possíveis que pode ser tomada nesses estados.

Então, temos um valor de para cada par de estado-ação, assim, para cada estado, ou seja, um conjunto de condições ao redor do agente, uma determinada ação terá um valor Q. Começa com um valor Q de zero para cada estado possível, à medida que explora, quando acontece algum coisa ruim reduzimos o valor Q do estado que ele se encontrava naquele momento, caso contrário, aumentamos esse valor. Esse processo de atualização pode ser descrito matematicamente com:

$$Q(s,a) += \text{desconto} * (recompensa(s,a) + \max(Q(s')) - Q(s,a))$$

, onde s é o estado anterior e s' o estado atual.

Q-Learning pode ser descrito como programação dinâmica, pois, é um método para resolver um problema complexo, dividindo-o em uma coleção de subproblemas mais simples — neste caso, encontrar o valor ótimo para cada par de estado-ação — armazenando as soluções na tabela Q e reutilizando-as sem precisar recalcular, economizando processamento.

2.2.4. Comparação entre eles

Algoritmo	Tipo de Aprendizado	Aplicação Prática na Aula	Principal Característica
k-NN	Supervisionado	Recomendação de Filmes (MovieLens)	Classifica por proximidade entre pontos
PCA	Não Supervisionado	Redução do dataset Iris de 4D para 2D	Preserva variância ao reduzir dimensões
Q-Learning	Por Reforço	Táxi autônomo (Taxi-v3)	Aprende por recompensas e penalidades

Tabela 1. Tabela comparativa dos algoritmos aprendidos em aula. (autoria própria)

2.3. Métricas de Avaliação de Modelos

As métricas são bem importantes para avaliar o quão bom o nosso modelo é, dentre elas existem métricas para modelos de classificação (Precisão, Recall, F1 Score, AUC) e métricas para modelos de regressão (RMSE).

Às vezes, a precisão não diz tudo, é preciso entender os verdadeiros positivos e verdadeiros negativos, bem como os falsos positivos e falsos negativos e algo que mostra isso é a matriz de confusão.

2.3.1. Matriz de Confusão

Ela é uma maneira de ilustrar as nuances da acurácia do modelo e normalmente vai seguir a seguinte aparência:

	Sim atual	Não atual
Sim previsto	Positivo verdadeiro	Positivo falso
Não previsto	Negativo falso	Negativo verdadeiro

Tabela 2. Matriz de confusão.

Em uma matriz real, essas células iriam conter os números reais de quantas vezes o modelo fez isso no *dataset* de teste. E não é toda matriz vai ser exatamente dessa forma, não há uma convenção para isso, então deve sempre prestar atenção aos rótulos.

Algo importante a observar também é que geralmente a maioria dos valores que queremos esteja aqui, a diagonal da matriz é onde a maioria dos resultados deve estar. Um modelo preciso teria os valores altos ao longo dessa diagonal.

Também pode haver outros formatos, por exemplo, para um modelo de multiclassificação seria algo parecido com essa:

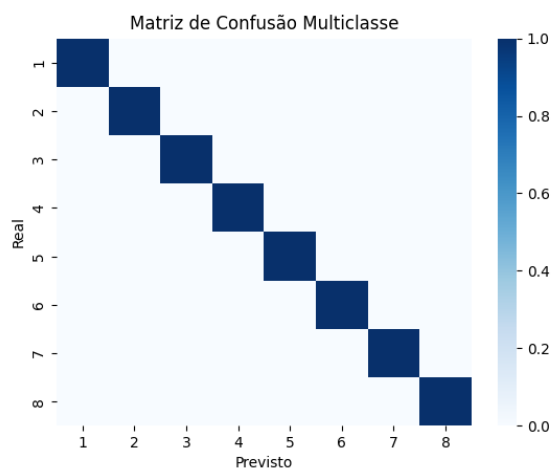


Figura 2. Matriz de confusão multiclasse feita com Seaborn e Matplotlib. (autoria própria)
Qual matriz usar vai depender do em que você está treinando.

2.3.2. Precisão

Ela é calcula através dessa fórmula:

$$Precisão = \frac{\text{Verdadeiros Positivos}}{\text{Verdadeiros Positivos} + \text{Falsos Positivos}}$$

Normalmente, pode ser chamada de taxa de acerto ou a porcentagem de resultados relevantes.

Ela é uma medida no mundo da recuperação de informações, utilizada normalmente quando se preocupa com os falsos positivos. Por exemplo, em exames médicos ou teste de drogas.

2.3.3. Recall

Ele é calculado da seguinte forma:

$$Recall = \frac{\text{Verdadeiros Positivos}}{\text{Verdadeiros Positivos} + \text{Falsos Negativos}}$$

Pode ser conhecida como sensibilidade, taxa de verdadeiros positivos ou completude. Ela é utilizada quando se preocupa com muito falsos negativos, detecção de fraudes é um bom

exemplo disso, porque um falso negativo no mundo da fraude significa que era fraude e o modelo não conseguiu identificar.

2.3.4. Especificidade e F1 Score

A especificidade é uma métrica para calcular a taxa de verdadeiros negativos, representada pela seguinte fórmula:

$$Especificidade = \frac{\text{Verdadeiros Negativos}}{\text{Verdadeiros Negativos} + \text{Falsos Positivo}}$$

E a F1 Score é média harmônica da precisão e da sensibilidade, no caso em que importa ter precisão e sensibilidade, F1 Score é a métrica que vai equilibrar as duas. Ela é calculada da seguinte forma:

$$F1\ Score = \frac{2 \cdot \text{Verdadeiros Positivos}}{2 \cdot \text{Verdadeiros Positivos} + \text{Falsos Positivos} + \text{Falsos Negativos}}$$

ou

$$F1\ Score = \frac{\text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}}$$

2.3.5. RMSE

É uma métrica usada frequentemente e é uma medida direta de acurácia. Soma todos os erros quadráticos de cada previsão. Afinal ele é utilizado para calcular a acurácia do modelo, a fórmula dele:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Onde y_i é o valor real, $\hat{y}_i$ é o valor previsto e n é o número de observações.

Aliás, RMSE é literalmente Raiz do Erro Quadrático Médio, claro que a sigla disso em inglês.

2.3.6. Curva ROC

A Curva ROC é uma maneira de avaliar os modelos (ROC é *Receiver Operating Characteristic*). Ela plota a taxa de verdadeiros positivos em várias configurações do seu modelo. Assim, conforme dá para diferentes limiares entre verdadeiro e falso.

A forma de interpretar ela é que você quer o que esteja acima da linha diagonal. Quanto mais curvada a curva ROC estiver em direção ao canto superior esquerdo, melhor.

2.3.7. AUC

É a área sob a curva ROC, ele é uma métrica para comparar classificadores, onde o maior valor é o melhor. Pode ser interpretado como a probabilidade do modelo classificar corretamente uma instância positiva acima de uma instância negativa escolhida aleatoriamente é de 0,5, o esperado é um AUC de 0,5 ou superior, se tiver uma linha diagonal. Se o AUC for menor de 0,5, é inútil ou pior.

2.4. Viés vs. Variância

Um dos principais desafios ao lidar com dados do mundo real é o sobreajuste e subajuste dos modelos ou das previsões. Quando se fala deles, dá para abordar sobre eles no contexto de viés e variância e a relação entre esses dois.

O Viés basicamente seria o quão distante está dos valores corretos.

A variância é uma medida de quão dispersas estão as previsões.

Um exemplo, imagine quatro situações:

1. Viés alto e variância alta: os acertos estão longe do centro e espalhado.
2. Viés baixo e variância alta: os acertos estão espalhados ao redor do centro.
3. Viés alto e variância baixa: os acertos estão próximos, mas longe do centro.
4. Viés baixo e variância baixa: os acertos estão agrupados e no centro.

Se o viés é baixo significa que as previsões estão espalhadas em torno do mesmo ponto e uma variância baixa significa que os pontos estão agrupados bem próximos, esse seria um mundo perfeito, mas na realidade, muitas vezes será preciso de um ou de outro.

Alto viés significaria um subajuste, ou seja, o modelo é muito simples e erra sistematicamente.

Alta variância significa um sobreajuste, onde o modelo está se adaptando demais aos dados de treino e não generaliza.

Porém abaixar o viés tende a aumentar a variância e vice-versa, o objetivo é encontrar um equilíbrio entre os dois.

2.5. Validação Cruzada K-Fold

A Validação Cruzada K-Fold é outra ferramenta para combater o sobreajuste, ela torna o uso de *datasets* de treino e teste melhor.

A ideia é treinar o modelo usando apenas os dados de treinamento e, em seguida, avaliar o desempenho usando dados reservados para o conjunto de teste, mas pode acabar sobreajustando os dados à divisão de treinamento e teste. Talvez o conjunto de treinamento não tenha sido representativo de todo o conjunto de dados, e muitas informações tenham sido incluídas nele, o que acaba distorcendo os resultados. Agora entra a Validação Cruzada K-Fold, ela aprimora o processo de divisão entre dados de treinamento e teste.

A ideia é simples, em vez de dividir em dois grupos, ela divide em k grupos, onde k é a quantidade de grupos, e separa um desses grupos para teste, onde os $k-1$ grupos restantes são usados para treinamento. Em seguida, é medido o erro quadrático médio resultante ao usar o grupo como dados de teste, depois, passa para o próximo grupo de k grupos e usa como dados de teste os $k - 1$ grupos restantes dos dados de treinamento.

Medimos os erros novamente e continuamos fazendo isso até usar todos os k grupos como conjunto de teste.

Ao final, após usar todos os k grupos, calcula-se a média dos erros de todos os k grupos, para obter uma estimativa mais confiável do desempenho real do modelo. O valor de k mais comum na prática é 10.

2.6. Limpeza de Dados e Normalização

Uma das tarefas mais importantes, acredito eu, e que mais vai consumir tempo é a limpeza dos dados. A qualidade da limpeza e a compreensão dos dados brutos terão um impacto enorme na qualidade dos modelos. Afinal, dados ruins resultam em um modelo ruim.

Os dados brutos que são recebidos, em sua maioria, sujos e poluídos de várias maneiras, se não lidar com eles, eles vão distorcer os resultados.

Existem diferentes tipos de problemas e dados com os quais é necessário ter cuidado:

- Outliers: valores discrepantes são coisas que não são boas de se manter no conjunto de treinamento, se estiver realizando alguma análise, não é bom descartá-los, se o

outlier for legítimo e relevante para a análise, mantê-lo pode ser importante; se for um erro ou ruído inconsistente com o que está sendo modelado, o ideal é descartá-lo.

- Dados ausentes: depende muito de que vou fazer, posso criar uma classificação para eles, se for numérico, posso calcular a média, para não alterar pela média geral, poderia usar k-NN para gerar esses dados ausentes com base nos vizinhos mais próximos, em último caso, se necessário, apagar eles. Vai depender muito do que eu achar que é certo fazer.
- Dados maliciosos: pode haver pessoas tentando manipular meu sistema, para isso posso tentar identificar esses ataques e filtrar os resultados.
- Dados errôneos: se houver um bug de software em algum sistema que esteja registrando valores incorretos em determinada situação, é difícil saber disso, mas se eu ver que parece suspeito, é bom investigar.
- Dados irrelevantes: ... Descartar eles.
- Dados inconsistentes: vai exigir de mim para descobrir quais são as variações e normalizá-las.
- Formatação: precisa ter cuidado e garantir que variações na formatação não sejam tratadas como entidades ou classificações diferentes durante o processamento.

Bom... Falei de alguns problemas comuns e algumas possíveis soluções para eles.

Se meu modelo for baseado em vários atributos numéricos, a primeira pergunta a se fazer é: São comparáveis?

Alguns modelos podem não ter um bom desempenho quando as escalas de um atributo são muito diferentes, algoritmos baseados em distância como o k-NN são especialmente sensíveis à escala dos dados, então é bom normalizar eles, para isso o Scikit-learn oferece uma opção para normalizá-los quando necessário, garantindo que os atributos sejam comparáveis entre si.

2.7. Engenharia de Recursos

A engenharia de recursos é o processo de aplicar o conhecimento dos dados e do modelo que estou usando para reduzir os recursos que estou usando ou criar recursos melhores para treinar o modelo.

Uma grande parte da engenharia de recursos envolve a imputação de dados

2.7.1. Imputação de Dados

A imputação de dados consiste em substituir os dados ausentes por outros, entre eles, envolve:

- Substituição pela média, que normalmente consiste em substituir os ausentes pela média da coluna ou pela mediana, que pode ser uma escolha melhor quando há *outliers*.
- Exclusão, mas em último caso, se houver muitas linhas com dados ausentes, se não enviesa os dados, se eu não tenho muito tempo, mas no geral é sempre bom não excluir dados.
- Aprendizado de máquina pode ser algo bom a se fazer, um algoritmo k-NN para calcular a média dos mais próximos no caso de tipos numéricos, para categóricos usar deep learning, embora seja mais complicado, ou por regressão, utilizando MICE.
- A abordagem mais eficaz, quando possível, é simplesmente obter mais dados.

2.7.2. Dados Desbalanceados

Quando se trata de dados desbalanceados, que normalmente ocorre quando há uma grande discrepância entre os casos de positivos e negativos, há maneira de lidar, como:

- *Oversampling* (sobreamostragem), consiste em pegar a classe minoritária e copiar elas até ficar próximo a classe majoritária
- *Undersampling* (subamostragem), ao contrário da primeira, ela consiste em remover a classe majoritária
- SMOTE, ela gera novas amostras da classe minoritária executando os algoritmos k-NN, o que é melhor que as duas primeiras opções
- Por último, a abordagem mais simples ajustar os limites.

2.7.3. Outras Técnicas

Outras técnicas que poderá utilizar são:

- Agrupamento: pegar dados numéricos e transformar em categóricos, agrupando com base em intervalos.
- Transformando: aplicar alguma função a um recurso para tornar mais adequado ao treinamento.
- Codificação: transformar os dados e codificá-los do formato que o modelo exige.
- Escalonamento e normalização: A maioria dos modelos exige que os dados sejam pelo menos escalonados para valores comparáveis, caso contrário, os recursos com magnitudes maiores terão mais peso do que deveriam.
- Embaralhamento: Não deixar na ordem que foi coletado.

3. Conclusão

Essas seções do curso abordaram como lidar com dados do mundo real para projetos de ML, desde a infraestrutura de *Data Warehousing* até a preparação de dados. Ficou claro que limpeza, normalização e engenharia de recursos são etapas essenciais para garantir a qualidade dos dados e, conseqüentemente, dos modelos. Saber usar as métricas de avaliação é fundamental, pois os resultados obtidos impactam diretamente o produto final. Compreender o trade-off entre viés e variância é essencial para treinar bem o modelo e conhecer o problema que está sendo resolvido. Por fim, em empresas menores, um cientista de dados tende a passar a maior parte do tempo realizando filtragem e limpeza de dados, o que reforça que dominar essas etapas é tão importante quanto conhecer os algoritmos.

Referências:

Curso **Machine Learning, Data Science and Generative AI with Python**:
<https://www.udemy.com/course/data-science-and-machine-learning-with-python-hands-on/>

Material do curso:

<https://www.sundog-education.com/machine-learning/>